Idea 1: API Change Radar

Problem: Dev teams depend on dozens of third-party APIs (Stripe, Twilio, GitHub, Shopify). Deprecations and breaking changes are announced in scattered changelogs, emails, and docs pages, and teams find out when production breaks.

Product: Users pick the APIs they use. You watch changelogs, docs, and OpenAPI specs, diff them, classify the changes (breaking, deprecation, new), and alert via email, Slack, or webhook.

Money: Free for 3 watched APIs, then paid tiers by number of watchers and alert channels. Teams pay readily for anything that prevents outages.

Learning: This one touches nearly everything you listed.

- Scheduled distributed crawling with a job queue and workers
- Idempotency, retries, and dead-letter queues
- Per-domain rate limiting and politeness
- Diffing and change detection
- Multi-tenancy and fan-out notifications
- Webhooks, including signing them
- **SSRF protection.** Your server fetches user-supplied URLs, which is a classic and serious vulnerability, so you'll learn it properly.

Suggested tech stack

Layer	Choice	Why
Backend	ASP.NET Core Web API (.NET 9), EF Core + Npgsql	Your preference, with a clean architecture
Database	Supabase Postgres	Free tier; use it as plain Postgres
Auth	ASP.NET Identity + JWT + refresh tokens (your own)	You learn far more than with a hosted auth service
Frontend	Next.js (React + TypeScript), Tailwind, shadcn/ui, TanStack Query	Deploys to Vercel with no friction
Cache and rate limits	Upstash Redis (free)	Rate limiting, distributed locks, caching
Queue	Start with a Postgres queue (`FOR UPDATE SKIP LOCKED` plus an outbox table), then move to RabbitMQ (CloudAMQP free)	Learn the fundamentals first, then the broker
Background jobs	.NET `BackgroundService` plus Quartz.NET	Scheduling and workers

Email	Resend (free tier)	Alerts and verification
Payments	Stripe test mode, or Lemon Squeezy/Paddle if Stripe isn't available in your country	Subscriptions, trials, webhooks
Observability	OpenTelemetry + Grafana Cloud (free)	Logs, traces, metrics
CI/CD	GitHub Actions, Docker	Automated builds and deploys

Free-tier caveats to know up front:

- Render's free web services sleep after about 15 minutes idle, and background workers aren't free. Run your worker inside the API process and wake it with a free pinger such as cron-job.org or GitHub Actions.
- Supabase free projects pause after inactivity.
- These limits are fine for learning and early validation. Budget for a small paid tier once real users arrive.

Learning roadmap

1. **Foundation:** Clean architecture, EF Core migrations, signup and login with password hashing, JWT and refresh token rotation, email verification, and the OWASP Top 10 applied to your own code.
2. **Core product:** Watch-target CRUD, a single-worker crawler, and diffing.
3. **Make it distributed:** Job queue, multiple workers, idempotent jobs, retries with backoff, dead-letter handling, per-domain rate limiting, and distributed locks.
4. **Harden security:** SSRF defenses (DNS and IP validation, blocking internal ranges), API rate limiting, secrets management, audit logs, and HMAC-signed outbound webhooks.
5. **Monetize:** Free trial logic, plan limits, Stripe checkout and webhooks, and usage metering.
6. **Production:** Observability, load testing with k6, caching, and database indexing and partitioning.

AI Integration: AI Upgrade for API Change Radar (easiest to add)

If you build the earlier idea, AI makes it much stronger.

- **Changelog parsing:** an LLM turns messy changelog prose into structured entries (breaking / deprecation / new, affected endpoints, deadline dates)
- **Impact analysis:** the user connects their repo (GitHub App, read-only), and the system finds where they call the deprecated endpoint and explains the fix
- **Migration summary:** "Stripe is retiring X on March 1. You use it in 3 files. Here's the change."